

Node.js Complete Guide

Author: Tech Learning

Chapter 1: Introduction to Node.js

Node.js is a JavaScript runtime built on Chrome's V8 JavaScript engine. It allows developers to run JavaScript on the server side. Node.js uses an event-driven, non-blocking I/O model that makes it lightweight and efficient. It is perfect for data-intensive real-time applications. Node.js was created by Ryan Dahl in 2009 and has since become one of the most popular server-side technologies.

Chapter 2: Event Loop

The event loop is the core mechanism that allows Node.js to perform non-blocking operations. Despite JavaScript being single-threaded, the event loop enables handling multiple concurrent operations. When an async operation completes, its callback is added to the event queue. The event loop continuously checks if the call stack is empty and processes queued callbacks.

Chapter 3: Async Programming

Node.js supports three patterns for async programming: callbacks, promises, and async/await. Callbacks were the original approach but led to callback hell. Promises provide a cleaner way to handle async operations with `.then()` and `.catch()` chains. `async/await` is syntactic sugar over promises that makes async code look synchronous. Always use `try/catch` blocks when using `async/await`.

Chapter 4: Express.js Framework

Express.js is the most popular web framework for Node.js. It provides a minimal and flexible set of features for web applications. Middleware functions are the core of Express, executing during the request-response cycle. Routes define how the application responds to client requests. Express supports template engines, static files, and error handling middleware.

Chapter 5: Security Best Practices

Security is critical in Node.js applications. Always validate and sanitize user input to prevent injection attacks. Use JWT tokens for stateless authentication in REST APIs. Implement rate limiting to prevent DDoS and brute force attacks. Use helmet.js to set secure HTTP headers. Store secrets in environment variables, never in code. Use HTTPS in production.

Chapter 6: Database Integration

Node.js works well with both SQL and NoSQL databases. MongoDB is a popular NoSQL choice that stores data as JSON-like documents. Mongoose provides an elegant ODM for MongoDB with schema validation. PostgreSQL is a powerful relational database for structured data. Use connection pooling to manage database connections efficiently. Always use parameterized queries to prevent SQL injection.

Chapter 7: Performance Optimization

Performance is key in production Node.js applications. Use clustering to utilize all CPU cores by forking multiple processes. Worker threads handle CPU-intensive tasks without blocking the event loop. Implement caching with Redis to reduce database load. Use streams for processing large files instead of loading them into memory. Profile your application with `node --prof` to identify bottlenecks.

Chapter 8: Testing

Testing ensures your Node.js application works correctly. Jest is the most popular testing framework for JavaScript. Write unit tests for individual functions and modules. Integration tests verify that components work together correctly. Use supertest to test Express HTTP endpoints. Aim for high code coverage but focus on critical paths. Mock external dependencies in tests.

Chapter 9: Deployment

Deploying Node.js applications requires careful planning. Use PM2 process manager to keep your app running and handle crashes. Docker containers package your app with all dependencies. Environment variables manage configuration across environments. Use CI/CD pipelines to automate testing and deployment. Monitor your application with proper logging using Winston or Pino.

Chapter 10: Microservices

Microservices architecture splits applications into small independent services. Each service handles a specific business function and can be deployed independently. Use REST APIs or message queues for inter-service communication. Docker and Kubernetes help orchestrate microservices at scale. Implement circuit breakers to handle service failures gracefully.